Zinter

Language Reference & Programming Guide

A complete guide to programming in the Zinter language

Chapter 1 — Introduction

Zinter is a small, statically-typed interpreted language designed to be executed by the **Zinterpreter**, a C-based virtual machine. Programs are written in plain-text files with the **.Zinter** extension. The language emphasises explicit, token-based syntax: every variable, array and matrix access is wrapped in **&** delimiters that carry both the type and the name of the datum. This makes the parser simple and deterministic while keeping programs readable once the token grammar is understood.

Zinter supports three primitive scalar types (**int**, **float**, **char**), one-dimensional arrays of all three types, two-dimensional matrices of all three types, and user-defined functions. Control flow (if / else / for / while) is available but currently being implemented in the interpreter. I/O is handled through a family of **print_** and **scan_** instructions.

Running a Zinter program

Compile the Zinterpreter from source with any C99-compatible compiler, then invoke it from the command line:

```
./Zinterpreter -dt myprogram.Zinter # debug ON
./Zinterpreter -df myprogram.Zinter # debug OFF
```

The interpreter first calls **format_code()** to tokenise the source into an internal instruction array, then **build_state()** to map all block boundaries (functions, if, for, while …), then executes the **#{ }** system block, and finally calls **parse()** starting from **__start**.

Chapter 2 — Token Grammar

Almost every datum in Zinter is referenced through a **token**. A token is a string of the form:

```
&TYPE_DESCRIPTOR&NAME&
```

The first field between the first pair of **&** characters describes the type and, for arrays and matrices, the index or dimension. The second field is the name of the variable, array or matrix. Tokens appear in declarations, assignments, print/scan calls, and as arguments to set_to_ functions.

2.1 Type prefix table

Prefix	Meaning	Example token
i	integer variable or integer literal index	&i;&myVar;&
n	integer numeric literal	&n;&42&
l	float variable	&l;&myFloat;&
c	char variable	&c;&myChar;&
k	char literal	&k;&A;&
s	char array (string) or char matrix	&s;[3]&myStr;&
i[idx]	integer array element at idx	&i;[2]&myArr;&
l[idx]	float array element at idx	&l;[0]&fArr;&
s[idx]	char array element at idx	&s;[1]&str;&
i[r][c]	integer matrix cell (row r, col c)	&i;[0][2]&mat;&
l[r][c]	float matrix cell	&l;[1][3]&fmat;&
s[r][c]	char matrix cell	&s;[2][0]&cmat;&
f	file (not yet implemented)	&f;&myfile.Zlib;&

2.2 Index resolution

Wherever an index appears (inside the square brackets) it can be one of three forms:

Index form	Meaning	Example
N (integer literal)	Direct numeric index	&i;[2]&arr;&
&i;&varname;&	Value of an int variable used as index	&i;[&i;&idx;&]&arr;&
rN (register)	Register N (advanced use)	&i;[r0]&arr;&

*Note: only **integer** variables may be used as indices. Using a float or char variable as an index is a runtime error.*

Chapter 3 — Declarations

All variables, arrays and matrices must be declared before use. Every declaration ends with a colon **:**.

3.1 Scalar variables

Syntax	Type	Notes
int_ &i;&name;&:	integer	Initialised to 0
int_ &l;&name;&:	float	Initialised to 0.0 — note prefix l not f
char_ &c;&name;&:	char	Initialised to '\0'

```
int_ &i&counter&:
int_ &l&price&:
char_ &c&letter&:
```

3.2 One-dimensional arrays

Syntax	Element type	Notes
int_ &i;[N]&name;&:	integer	N cells, all 0
int_ &l;[N]&name;&:	float	N cells, all 0.0
char_ &s;[N]&name;&:	char	N cells, all '\0'

```
int_ &i[10]&scores&:
int_ &l[4]&prices&:
char_ &s[32]&name&:
```

N may be an integer literal or an integer variable token. Maximum array size is 64 cells.

3.3 Two-dimensional matrices

Syntax	Element type	Notes
int_ &i;[R][C]&name;&:	integer	R rows × C cols, all 0
int_ &l;[R][C]&name;&:	float	R rows × C cols, all 0.0
char_ &s;[R][C]&name;&:	char	R rows × C cols, all '\0'

```
int_ &i[4][4]&grid&:
int_ &l[3][8]&table&:
char_ &s[5][5]&board&:
```

Chapter 4 — Assignment

Zinter provides two syntaxes for assignment: a **compact operator syntax** (using **=**) and an explicit **set_to_** family of functions. Both are supported and produce identical results. Every assignment ends with **:**.

4.1 Compact operator syntax

The left-hand side is the destination, the right-hand side the source. Source and destination must have compatible types.

Pattern	Description	Example
`var = N`	literal → variable	`int_var0 = 7:`
`var = var2`	variable → variable	`int_var1 = int_var0:`
`var = 'ch'`	char literal → char variable	`char_var0 = 'H':`
`[i]arr = N`	literal → array cell	`[0]int_arr_5 = 9:`
`[i]arr = var`	variable → array cell	`[1]int_arr_5 = int_var0:`
`var = [i]arr`	array cell → variable	`int_var1 = [0]int_arr_5:`
`[i]arr = [j]arr2`	array cell → array cell	`[3]arr = [0]arr2:`
`[r][c]mat = N`	literal → matrix cell	`[0][0]int_mat = 42:`
`[r][c]mat = var`	variable → matrix cell	`[2][0]int_mat = int_var0:`
`var = [r][c]mat`	matrix cell → variable	`int_var1 = [0][0]int_mat:`
`[r][c]mat = [r2][c2]mat2`	matrix → matrix	`[1][0]m = [0][1]m:`
`[r][c]mat = [i]arr`	array cell → matrix cell	`[0][1]mat = [1]arr:`
`[i]arr = [r][c]mat`	matrix cell → array cell	`[0]arr = [0][0]mat:`

Indices may be integer variable names (resolved at runtime) instead of literals:

```
row = 1:
col = 2:
[row][col]int_mat = 99:
int_var1 = [row][col]int_mat:
```

4.2 set_to_ family

The explicit set_to_ functions are the older, lower-level API. They require a placeholder @ for unused type slots (int value when setting a char, or vice-versa).

set_to_variable_

```
set_to_variable_ name,type,value,cvalue:
```

```
# Examples
set_to_variable_ int_var0,i,&n&7&,@: # int <- 7
set_to_variable_ int_var1,i,&i&int_var0&,@: # int <- int variable
set_to_variable_ char_var0,c,@,&k&H&: # char <- literal 'H'
set_to_variable_ char_var1,c,@,&c&char_var0&: # char <- char variable
```

set_to_array_

```
set_to_array_ name,type,index,value,cvalue:
set_to_array_ int_arr_5,i,&n&0&,&n&9&,@: # int_arr_5[0] <- 9
set_to_array_ int_arr_5,i,&n&1&,&i&int_var0&,@: # int_arr_5[1] <- int_var0
set_to_array_ char_arr_5,s,&n&0&,@,&k&L&: # char_arr_5[0] <- 'L'
set_to_array_ char_arr_5,s,&n&1&,@,&c&char_var0&: # char_arr_5[1] <- char_var0
```

set_to_matrix_

```
set_to_matrix_ name,type,row,col,value,cvalue:
set_to_matrix_ int_mat,i,&n&0&,&n&0&,&n&42&,@: # int_mat[0][0] <- 42
set_to_matrix_ char_mat,s,&n&1&,&n&1&,@,&k&Z&: # char_mat[1][1] <- 'Z'
```

Chapter 5 — Input / Output

Zinter provides four print variants and one scan instruction. All I/O statements end with **:**.

5.1 Print family

Instruction	Behaviour
`print_ TOKEN:`	Print the value without a newline
`println_ TOKEN:`	Print the value, then a newline
`lnprint_ TOKEN:`	Print a newline, then the value
`lnprintln_ TOKEN:`	Print a newline, the value, then another newline

Printing different types:

```
print_ &i&myInt&: # prints integer variable
print_ &l&myFloat&: # prints float variable
print_ &c&myChar&: # prints char variable
print_ &n&99&: # prints literal 99
print_ &s&"hello world"&: # prints the string hello world
print_ &s&&: # prints a space
print_ &i[2]&myArr&: # prints array element [2]
print_ &l[0]&fArr&: # prints float array element [0]
print_ &s[1]&str&: # prints char array element [1]
print_ &i[0][1]&mat&: # prints matrix int cell [0][1]
print_ &s[2][2]&cmat&: # prints matrix char cell [2][2]
```

Short-form (without explicit token):

```
print_ myInt: # auto-resolves type
print_ [2]myArr: # array element
print_ [0][1]mat: # matrix cell
```

5.2 Scan (input)

scan_ reads a single character or integer from stdin into the specified target. Implementation is currently a stub; full implementation is forthcoming.

```
scan_ &c&myChar&:
scan_ &i&myInt&:
scan_ &s[0]&str&: # read into array element
scan_ &s[]&str&: # read entire string
```

Chapter 6 — Functions

Functions are declared with the **vd**_ prefix and called with the __ prefix. The main entry point of every Zinter program is the special function __**start**.

6.1 Declaration

```
vd_ functionName(arg1, arg2){
// body instructions
return &i&someVar&:
}
```

The argument list is currently positional. Return values are declared with **return** followed by one or more tokens. Multiple return values are separated by spaces with a **-** prefix per value.

6.2 Calling a function

Pattern	Description
`__name(args):`	Call with no returned value captured
`var = __name(args):`	Call, capture single return into var
`__name(args) --&i;&v1;& --&i;&v2;&:`	Call, capture multiple returns

```
// No return value
__calcola(arg1,arg2):
// Single return → variable
int_var0 = __calcola(arg1,arg2):
// Multiple returns
__calcola(a,b) --&i&result1& --&i&result2&:
```

6.3 The __start function

__**start** is the mandatory entry point. Execution begins here after the system block has been processed. It may receive arguments from the command line in future versions.

```
__start(){
int_ &i&x&:
x = 5:
println_ &i&x&:
}
```

Chapter 7 — The System Block #{ }

The system block is a special section delimited by **#{ }**. It is always executed before the main program. It is used to configure the interpreter and import libraries. Its instructions use a different, underscore-separated mini-language.

```
#{
import_ &f&myLibrary.Zlib&:
debug_ -dt:
exec_:
}
```

Directive	Effect
`import_ &f;&file.Zlib;&:`	Import a library file (not yet implemented)
`debug_ -dt:`	Enable debug output from this point
`debug_ -df:`	Disable debug output
`exec_:`	Transfer execution to `__start` — must be last

*The **exec_:** directive is mandatory. Without it the interpreter will not jump to __**start** and the program body will not execute.*

Chapter 8 — Comments

Single-line comments begin with **//**. Everything on that line (after the colon that terminates the previous instruction) is ignored by the parser.

```
// This is a comment:
int_ &i&x&: // inline comment after instruction
```

*There is no multi-line comment syntax. Use multiple **//** lines.*

Chapter 9 — Control Flow (in progress)

The following constructs are parsed and their block boundaries are registered by the interpreter, but full execution logic is currently being implemented.

9.1 if / else

```
if_( == &i&x& &n&0& ){
// executed when x == 0
}
else_ {
// executed otherwise
}
```

Comparison operators inside the condition:

Token	Meaning
==	equal
!=	not equal
>	greater than
<	less than
>=	≥
<=	≤

9.2 for

```
for_( < &i&i& &n&10&, + &i&i& &n&1& ){
// loop body — while i < 10, increment i by 1
}
```

9.3 while

```
while_( != &i&flag& &n&0& ){
// loop body — while flag != 0
}
```

Chapter 10 — Complete Example Program

The program below demonstrates declarations, assignments with the compact syntax, matrix operations and output.

```
#{
debug_ -df:
exec_:
}
vd_ greet(arg){
println_ &s&"Hello from Zinter!"&:
}
__start(){
// --- scalar variables ---
int_ &i&x&:
int_ &i&y&:
char_ &c&ch&:
int_ &l&pi&:
x = 10:
y = x:
ch = 'Z':
print_ &i&x&: println_ &s&" <- x"&:
print_ &i&y&: println_ &s&" <- y (copy of x)"&:
print_ &c&ch&: println_ &s&" <- ch"&:
// --- array ---
int_ &i[3]&arr&:
[0]arr = 11:
[1]arr = 22:
[2]arr = x:
print_ &i[2]&arr&: println_ &s&" <- arr[2] (=x)"&:
// --- matrix ---
int_ &i[2][2]&mat&:
[0][0]mat = 1:
[0][1]mat = 2:
[1][0]mat = 3:
[1][1]mat = 4:
print_ &i[1][1]&mat&: println_ &s&" <- mat[1][1]"&:
// --- function call ---
__greet(arg):
}
```

Chapter 11 — Quick Reference Card

Declarations

Statement	Declares
int_ &i;&name;&:	int variable
int_ &l;&name;&:	float variable
char_ &c;&name;&:	char variable
int_ &i;[N]&name;&:	int array of N
int_ &l;[N]&name;&:	float array of N
char_ &s;[N]&name;&:	char array of N
int_ &i;[R][C]&name;&:	int matrix R×C
int_ &l;[R][C]&name;&:	float matrix R×C
char_ &s;[R][C]&name;&:	char matrix R×C

Assignment (compact syntax)

Example	Effect
var = 7:	int literal to var
var = 'A':	char literal to char var
var2 = var1:	copy variable
[i]arr = 5:	literal to array cell
var = [i]arr:	array cell to variable
[r][c]mat = var:	variable to matrix cell
var = [r][c]mat:	matrix cell to variable
[r][c]m = [i]arr:	array cell to matrix cell

Output

Instruction	Effect
print_ &i;&v;&:	print int var
println_ &s;&"text"&:	print string + newline
lnprint_ &c;&v;&:	newline then char
print_ &n;&42&:	print literal number